

## **PurpleMind: Network Security and Machine Learning Laboratory**

PurpleMind is a Python project for studying network traffic, cybersecurity and Machine Learning. The project has one main system: **PurpleTeam**. PurpleTeam manages the complete process:

**Network Traffic** → **Dataset** → **Data Analysis** → **Machine Learning** → **Evaluation**

The project is made for learning and experimentation in a controlled environment.

### **Project Goal**

The main goal is to understand how Machine Learning can be used to analyze network events.

The project can:

- generate simulated network events;
- analyze network traffic;
- create a dataset;
- study the data;
- train Machine Learning models;
- compare different models;
- measure detection performance.

### **Project Structure**

The project is divided into modules.

#### **config.py**

Contains the main configuration values. For example:

- number of normal events;
- number of attack events;
- random seed;
- other simulation parameters.

This makes the project easier to change.

#### **traffic.py**

Manages network traffic. It can create controlled simulated scenarios with Scapy. It can also analyze authorized real network captures with PyShark.

### **dataset.py**

Creates and prepares the dataset. It converts network events into a Pandas DataFrame.

### **eda.py**

Contains Exploratory Data Analysis. It is used to understand the dataset with statistics and graphs.

### **models.py**

Contains the Machine Learning models. The project uses:

- Logistic Regression;
- Random Forest;
- Isolation Forest.

### **evaluation.py**

Evaluates the models. It calculates:

- Accuracy;
- Precision;
- Recall;
- Confusion Matrix;
- False Positives;
- False Negatives.

### **main.py**

Runs the complete project pipeline.

### **pteamLab.ipynb**

The notebook is used to study the project step by step. The notebook and main.py use the same general pipeline, but they have different purposes. main.py runs the process automatically. The notebook helps the user understand each step.

### **Libraries**

The main Python libraries are:

- pandas
- numpy
- scikit-learn
- matplotlib

- seaborn
- scapy
- pyshark

Python standard libraries are also used.

## **Installation**

Install the required libraries with:

```
python -m pip install pandas numpy scikit-learn matplotlib seaborn scapy  
pyshark
```

## **Machine Learning Models**

### **Logistic Regression**

Logistic Regression is the baseline model. It is simple and useful as a first reference.

### **Random Forest**

Random Forest is a more complex model. It uses many decision trees and can learn more complex patterns.

### **Isolation Forest**

Isolation Forest is used for anomaly detection. It is different from the supervised classification models. It tries to find unusual events in the data.

## **Dataset**

The dataset contains network event features such as:

- timestamp;
- source IP;
- destination port;
- bytes sent;
- duration;
- failed logins;
- request rate;
- label.

The label can be:

- benign
- malicious

For Machine Learning, these labels are converted into numbers:

- benign = 0
- malicious = 1

### **Important Concept: Overlap**

The dataset should not be too perfect. Normal and malicious events can have similar values. This is called **overlap**. Overlap makes the experiment more interesting because the Machine Learning model cannot use one simple rule to separate all events.

### **Model Evaluation**

The project uses several metrics:

#### **Accuracy**

Shows the percentage of correct predictions.

#### **Precision**

Shows how many positive predictions are correct.

#### **Recall**

Shows how many real positive events are detected.

Recall is especially important in security because missing an attack can be a serious problem.

#### **False Positive**

The model says that an event is malicious, but it is actually benign.

#### **False Negative**

The model says that an event is benign, but it is actually malicious.

False Negatives are very important in security because they represent events that were not detected.

### **Network Traffic**

PurpleMind can work with two types of traffic.

### **Simulated Traffic**

The project can create controlled events for the laboratory.

This is useful because we know the expected label.

## **Real Traffic**

PyShark can be used to analyze authorized network captures. Real traffic is more complex. A captured packet does not automatically have a malicious or benign label. Therefore, real traffic needs reliable labeling before it can be used correctly for supervised Machine Learning.

## **Safety**

Real network capture must only be performed on networks and systems where you have permission. For learning, it is recommended to use:

- your own computer;
- a virtual machine;
- a private laboratory network;
- authorized network captures;
- public datasets with appropriate licenses.

The project does not require real attacks for Machine Learning training. Simulated data is enough for the first experiments.

## **Learning Path**

A good way to study the project is:

1. Read config.py.
2. Study traffic.py.
3. Study dataset.py.
4. Study eda.py.
5. Study models.py.
6. Study evaluation.py.
7. Finally study main.py.
8. Use pteamLab.ipynb to repeat the experiments.

The important idea is to understand the complete process, not only the Machine Learning model.

## **Skills Learned**

PurpleMind connects different technical areas.

## **Python**

- modular programming;

- functions;
- imports;
- data structures;
- file management;
- libraries.

## **Networking**

- IP addresses;
- ports;
- packets;
- network traffic;
- traffic analysis.

## **Cybersecurity**

- network monitoring;
- detection;
- false positives;
- false negatives;
- anomaly detection.

## **Machine Learning**

- datasets;
- features;
- labels;
- training;
- testing;
- classification;
- anomaly detection;
- model evaluation.

## **Data Science**

- Pandas;
- NumPy;
- data cleaning;
- statistics;
- EDA;
- visualization.

## **The Main Idea**

PurpleMind connects programming and networking with Machine Learning. This project is a starting point for learning how a small network security detection system can be designed.

## **Future Improvements**

Possible future improvements are:

- dynamic configuration;
- more realistic traffic;
- more difficult datasets;
- automatic retraining;
- concept drift;
- ensemble models;
- real-time monitoring;
- SOC dashboard;
- MITRE ATT&CK mapping;
- graph-based network analysis;
- event streaming.

The project should be improved step by step, keeping the system understandable and controlled.